

Can a Computer Learn From Its Mistakes?

How Artificial Intelligence (AI)
Gets Better at Guessing

PRESENTED BY

Esther Odaibo

King Owusu

Louis Delgado

Zain Ahmed



A mistake is a place to start.



How Does a Computer Know When It's Wrong?

People label practice photos “cat” or “dog.” The labels are an answer key, not clues for the guess.

1 INPUT

“Look at the picture”



The computer gets a photo. The answer label stays hidden for now.

2 FORWARD PASS

“Make a guess”



Using its current settings, the network makes a guess: “Dog?”

3 LOSS CALCULATION

“Check the answer”



The label says “cat.” Loss is an error score: bigger means more wrong.

4 WEIGHT UPDATE

“Adjust the controls”



Like volume controls, weights make signals count more or less. They are number settings, not real knobs. The computer adjusts them to try to lower its error score.

5 NEXT ITERATION

“Practice again”



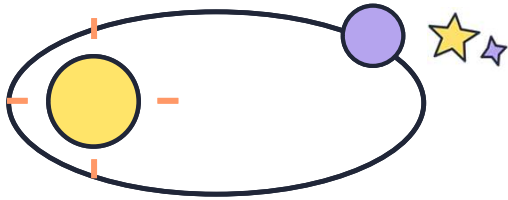
Try more labeled photos with the new settings. Each practice round is an iteration.

Big question: Which controls should we adjust, and by how much?

From Space Orbits to “Cat or Dog?”

One old idea connects them: use slope information to improve a guess.

1847 / CAUCHY’S CHALLENGE



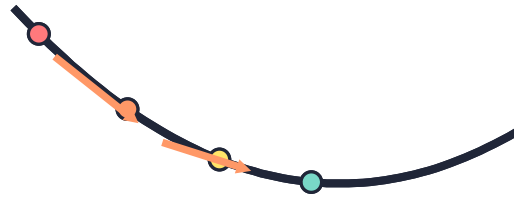
Augustin-Louis Cauchy

Orbit calculations were hard!
Cauchy developed a method
to improve estimates using
the slope of an error score.

Long before AI!

GRADIENT / THE SLOPE CLUE

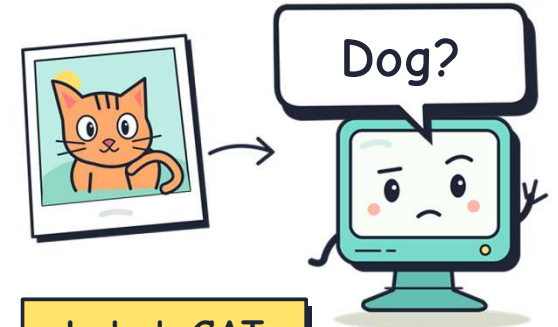
HIGHER ON THIS MAP = MORE ERROR



Gradient: slope information
that points uphill.

Descent: step the other way
to try to lower the error.
Check the slope again.

AI / THE SAME IDEA



Label: CAT

Same cat. Same answer key.
AI uses the slope to change
its weights, the settings that
influence its guesses.

Space or cats: check the slope, then step toward less error.

Help the Computer Find "Cat"

Same photo. Same CAT answer key. Adjust the controls, not the cat!

LOSS LANDSCAPE / A MAP OF POSSIBLE SETTINGS

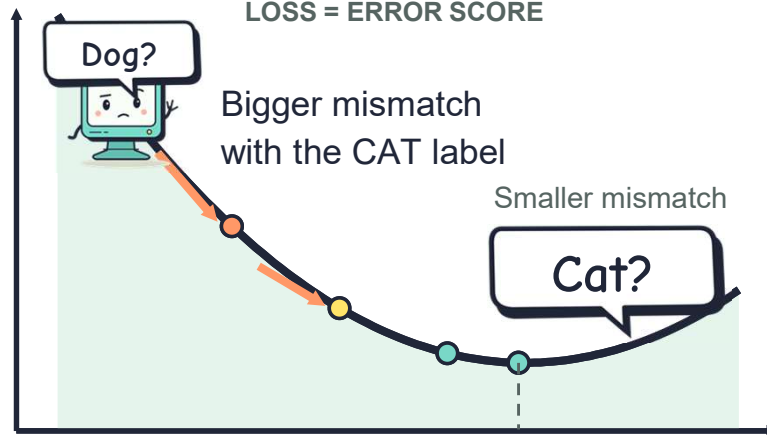
ANSWER KEY



CAT

This photo and label stay put.

LOSS = ERROR SCORE

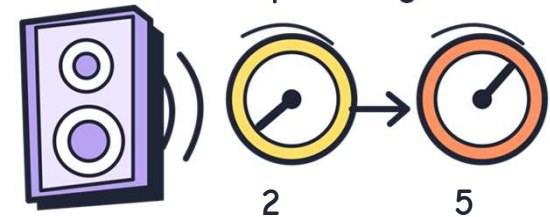


WEIGHT = ONE CONTROL SETTING

Move the setting. Try to lower the loss.

WEIGHTS / "CONTROL SETTINGS"

Like speaker volume, weights change how strongly signals influence the computer's guess.



Example settings, not actual cat / dog weights.

LEARNING RATE / "STEP SIZE"

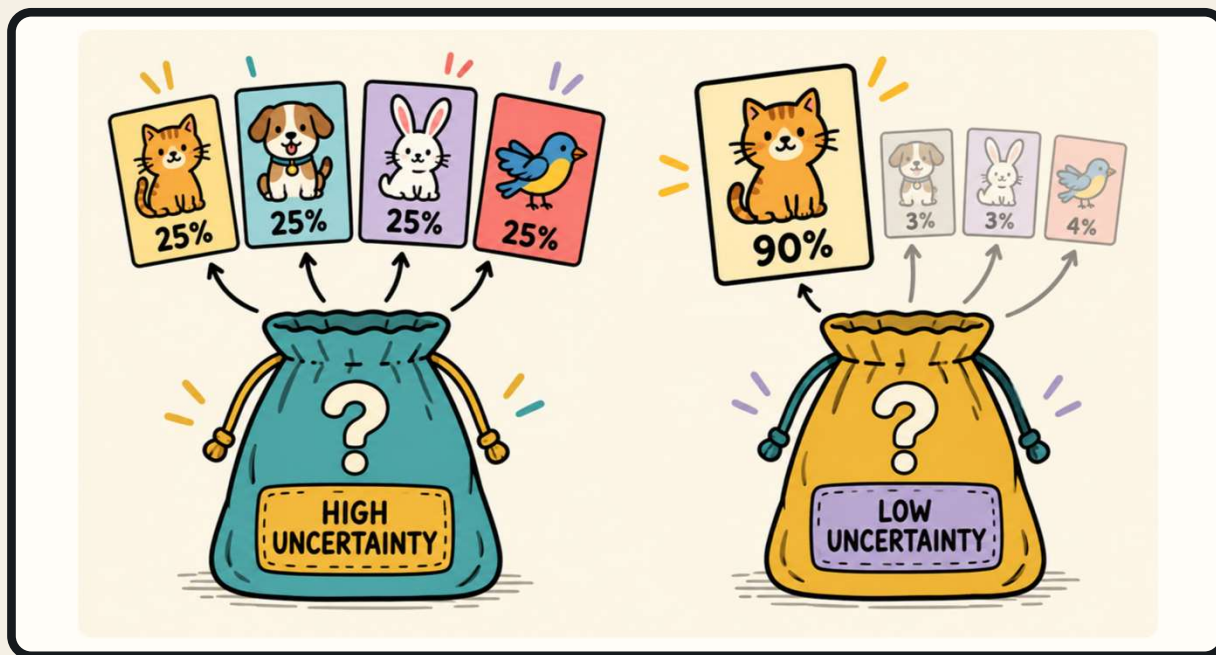
Tiny steps can be slow.
Huge jumps may overshoot.

Going downhill means smaller loss, not always smaller weights!

Illustrative example. Sources: Google Machine Learning Crash Course and Google Codelabs. Details in notes.

Entropy: How Unsure Is the Computer?

Entropy turns the computer's uncertainty into a number.



THE REAL IDEA

The computer gives every possible answer a chance.

Similar chances = high entropy
One clear favorite = low entropy

WHY IT MATTERS

It shows whether the model feels certain or unsure before it chooses a label.

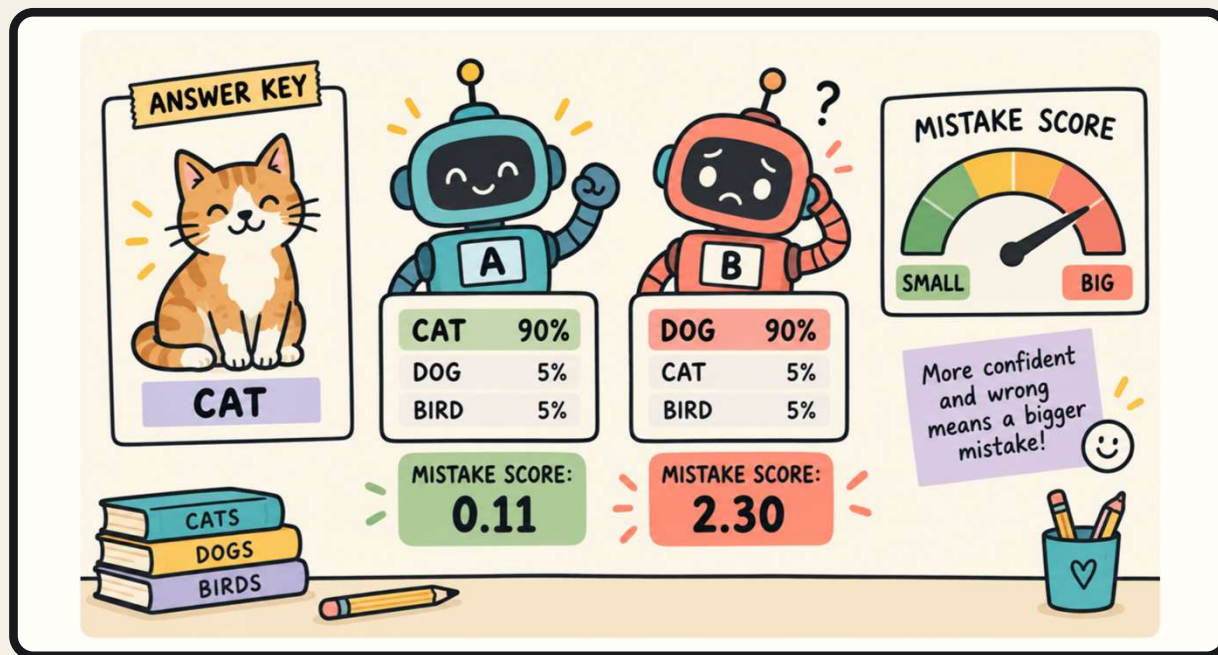
WHERE THE ANALOGY STOPS

A real network does not feel confused. It calculates entropy from probabilities.

Quick check: Which bag has greater entropy, and what makes it greater?

Cross Entropy: The Mistake Score

Cross entropy compares the prediction with the correct answer.



WHAT PROBLEM DOES IT SOLVE?

It measures how far the predicted chances are from the correct label.

THE REAL RULE

Loss = $-\log(\text{chance given to the correct answer})$

High chance for the correct answer gives a small loss. A confident wrong guess gives a large loss.

HOW LEARNING USES IT

Training tries to lower this score. Backpropagation then helps decide which weights to adjust.

Analogy limit: The score measures the mistake. It does not choose the weight changes by itself.

Backpropagation: Learning From Mistakes

Backpropagation is a method that helps a neural network learn from its mistakes.

1 SHOW A PICTURE

"It's a cat!" 🐱



The network is shown a photo of a cat. It has no idea yet what the right answer is.

2 MAKE A GUESS

"Dog?" ❌



Using its current settings, the network predicts "Dog" — and that guess is wrong.

3 FIND THE MISTAKE

"How wrong?" 📊



The real answer is "cat." The network scores how wrong that guess was.

4 GO BACKWARD

"Send the error back" ⬅️ BACK

That error score travels backward through the network, layer by layer — showing each part how much it contributed to the mistake.

5 FIX THE WEIGHTS

"Adjust it" 🔧



The network figures out which weights to change, so it guesses better next time.

Key takeaway: Backpropagation helps a neural network understand what contributed to its error.

The Learning Loop

Backpropagation repeats this cycle again and again until the network gets better.

1 INPUT

"Data goes in" 



Raw input — like a photo — enters the neural network to begin the loop.

2 NEURAL NETWORK

"Process it" 



The network passes the input through its layers and comes up with a prediction.

3 PREDICTION → ERROR

"Spot it" 



The prediction is compared to the answer to score how wrong it was.

4 BACKPROPAGATION

"Send it backward" 

The error travels backward through the network, and the network calculates gradients — directions that show how to reduce the error.

5 ADJUST & REPEAT

"Try again!" 



The network adjusts its weights using the gradients, then tries again!



The loop: Input → Predict → Error → Backprop → Adjust → Repeat!

The Building Blocks of a Smart Guess

A neural network uses weights, a bias, and an activation function to turn information into a decision.

1. WEIGHTS

How important is this feature?



Weights are numbers that make some features count more or less. If a feature is important (like having 4 legs for a cat), it gets a bigger weight.

Bigger weight =
bigger influence!

2. BIAS

A little push up or down.

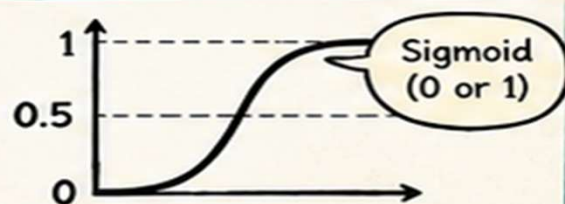


The bias is a number that shifts the total score. It helps the model be more flexible, even if the input features are small.

Think of it as a
starting boost!

3. ACTIVATION FUNCTION

Should the neuron react or not?



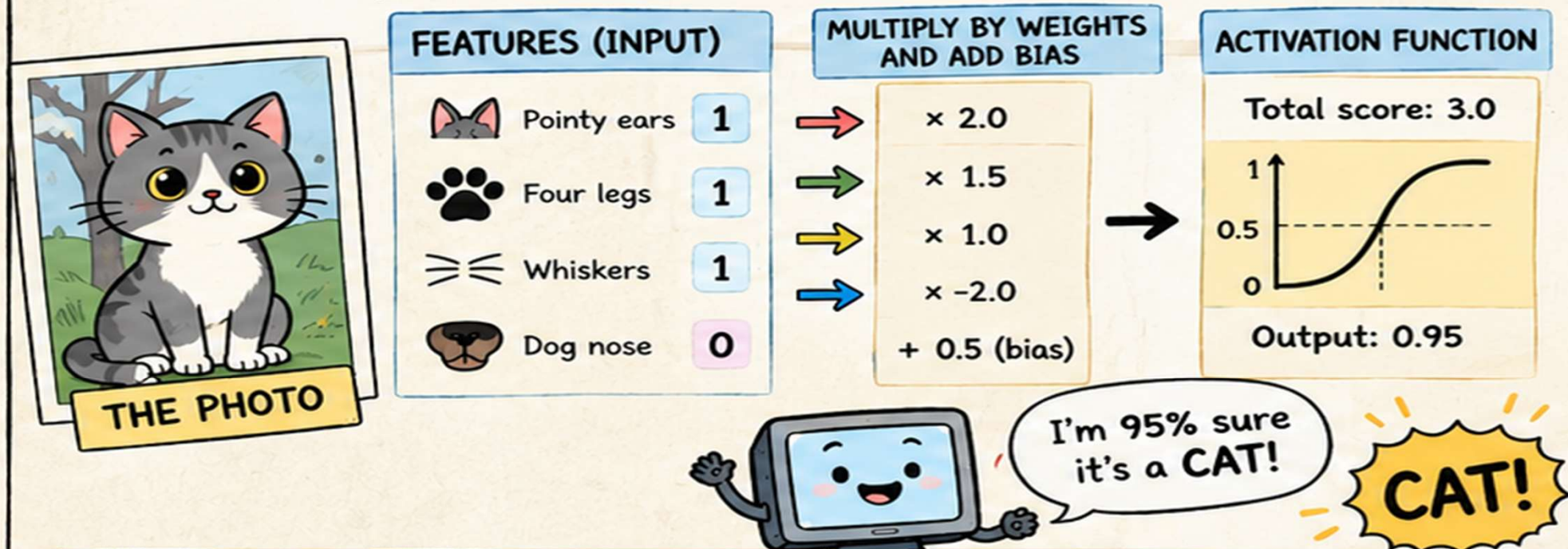
The activation function takes the total score and squashes it into a useful range (like 0 to 1). It helps the model decide how confident it is.

Turns a number into
a decision!

Weights choose what to listen to. Bias gives a little push.
The activation function decides what to do next!

Can the Computer Spot a Cat?

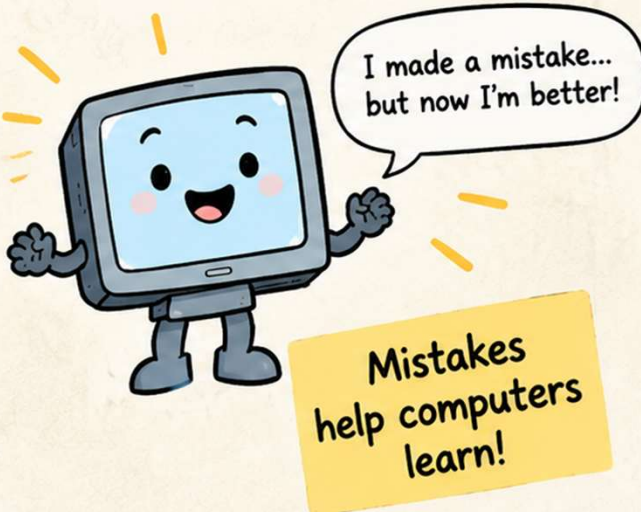
Let's see how weights, bias, and an activation function work together on a real example!



The weights say which features matter, the bias gives a little push, and the activation function turns the score into a confident guess!

So... Can a Computer Learn From Its Mistakes?

Yes! And now you know how.



We use practice examples (like our cat and dog photos).



The computer makes a guess using weights, bias, and an activation function.



It checks how wrong it was using a loss score.



It uses gradient descent to adjust the weights and bias.



It practices again and again, getting better each time!



Deep learning isn't magic. It's practice, feedback, and small improvements that add up to big results.

